

Laboratorio 3 Cloud Computing (15%)

UDEA - 2026

Laboratorio 3 — CRUD Serverless con AWS Lambda, API Gateway y DynamoDB

Objetivo general

Construir una API REST completamente serverless que permita crear, consultar, actualizar y eliminar registros (CRUD), usando funciones AWS Lambda expuestas por API Gateway y persistiendo la información en una tabla de Amazon DynamoDB, todo definido como infraestructura como código con Serverless Framework.

Objetivos específicos

- Definir en un único archivo `serverless.yml` las funciones, los endpoints HTTP, la tabla de DynamoDB y los permisos IAM.
- Diseñar la llave primaria (partition key) de una tabla DynamoDB y entender el modelo clave-valor.
- Implementar una función Lambda por cada operación del CRUD usando el AWS SDK v3.
- Aplicar el principio de mínimo privilegio: cada servicio solo puede ejecutar las acciones que necesita sobre su tabla.
- Manejar correctamente los códigos de respuesta HTTP (200, 201, 400, 404, 500) y la validación de datos.
- Probar el servicio localmente con `serverless-offline` y en AWS con Postman o Insomnia.
- Verificar los recursos creados en la consola de AWS (Lambda, API Gateway, DynamoDB y CloudWatch).

Descripción del laboratorio

Cada grupo debe construir una API para administrar una entidad de negocio. En esta guía se usa **productos** como ejemplo, pero **cada grupo debe elegir una entidad propia** (por ejemplo: libros, estudiantes, citas médicas, reservas, vehículos, eventos) con **al menos 4 atributos** además del identificador.

La API debe exponer los siguientes endpoints, cada uno atendido por su **propia función Lambda**:

Método	Ruta	Función	Respuesta esperada
POST	/productos	crear	201 con el registro creado · 400 si los datos no son válidos
GET	/productos	listar	200 con la lista de registros
GET	/productos/{id}	obtener	200 con el registro · 404 si no existe

Método	Ruta	Función	Respuesta esperada
PUT	/productos/{id}	actualizar	200 con el registro actualizado · 400 · 404
DELETE	/productos/{id}	eliminar	200 con mensaje de confirmación · 404

Arquitectura

Cliente (Postman / Insomnia) → API Gateway (HTTP API) → AWS Lambda (5 funciones) → Amazon DynamoDB (1 tabla)

Los logs de cada función quedan en Amazon CloudWatch. Todo se crea con un solo `serverless deploy`, que por debajo genera una pila de AWS CloudFormation.

Requisitos técnicos

- Node.js 20 o superior y Serverless Framework v4 (`npm install -g serverless`).
- Cuenta en `app.serverless.com` (organización) y cuenta activa de AWS.
- Credenciales de AWS configuradas (`aws configure` o proveedor conectado en el dashboard de Serverless) con permisos sobre Lambda, API Gateway, DynamoDB, IAM, CloudFormation, S3 y CloudWatch Logs.
- Plugin `serverless-offline` para pruebas locales.
- Postman o Insomnia para las pruebas.

Pasos sugeridos para el desarrollo

1. Crear el proyecto

Ejecutar `serverless` y seleccionar la plantilla **AWS / Node.js / HTTP API**. Nombrar el servicio, por ejemplo, `crud-productos`. Luego instalar las dependencias:

```
serverless
cd crud-productos
npm install @aws-sdk/client-dynamodb @aws-sdk/lib-dynamodb
npm install serverless-offline --save-dev
```

2. Configurar el `serverless.yml`

El archivo debe definir:

- Organización, aplicación y nombre del servicio.
- Runtime, arquitectura y región.
- Variable de entorno con el nombre de la tabla (las funciones no deben tener el nombre quemado en el código).
- Permisos IAM limitados a la tabla del servicio.
- Las cinco funciones con sus eventos `httpApi`.
- La tabla DynamoDB en la sección `resources` (sintaxis CloudFormation).
- El plugin `serverless-offline`.

Ejemplo (Node.js):

```
# Reemplazar "org" por la organización de su cuenta en app.serverless.com
org: universidaddeantioquia
app: crud-productos
```

```
service: crud-productos

provider:
  name: aws
  runtime: nodejs24.x
  architecture: arm64
  region: us-east-1
  environment:
    PRODUCTOS_TABLE: ${self:service}-productos-${sls:stage}
  iam:
    role:
      statements:
        - Effect: Allow
          Action:
            - dynamodb:PutItem
            - dynamodb:GetItem
            - dynamodb:Scan
            - dynamodb:UpdateItem
            - dynamodb>DeleteItem
          Resource:
            - Fn::GetAtt: [ProductosTable, Arn]

functions:
  crear:
    handler: handler.crear
    events:
      - httpApi:
          path: /productos
          method: post
  listar:
    handler: handler.listar
    events:
      - httpApi:
          path: /productos
          method: get
  obtener:
    handler: handler.obtener
    events:
      - httpApi:
          path: /productos/{id}
          method: get
  actualizar:
    handler: handler.actualizar
    events:
      - httpApi:
          path: /productos/{id}
          method: put
  eliminar:
    handler: handler.eliminar
    events:
      - httpApi:
          path: /productos/{id}
          method: delete

resources:
  Resources:
    ProductosTable:
      Type: AWS::DynamoDB::Table
      Properties:
        TableName: ${self:provider.environment.PRODUCTOS_TABLE}
        BillingMode: PAY_PER_REQUEST
        AttributeDefinitions:
          - AttributeName: id
            AttributeType: S
        KeySchema:
          - AttributeName: id
            KeyType: HASH

plugins:
```

Notas:

- `${sls:stage}` hace que la tabla se llame distinto en cada ambiente (dev, prod), evitando que se mezclen los datos.
- `PAY_PER_REQUEST` (modo bajo demanda) no cobra capacidad reservada: ideal para el laboratorio.
- `Fn::GetAtt: [ProductosTable, Arn]` obtiene el ARN de la tabla creada, de modo que las Lambdas solo pueden operar sobre esa tabla.

3. Implementar la lógica del CRUD

Archivo: `handler.js`. Se comparte un cliente de DynamoDB (creado fuera de los handlers para reutilizarlo entre invocaciones), una función para armar las respuestas HTTP y una validación común. A continuación se entregan las operaciones **crear**, **listar** y **obtener**:

```
const { DynamoDBClient } = require("@aws-sdk/client-dynamodb");
const {
  DynamoDBDocumentClient,
  PutCommand,
  GetCommand,
  ScanCommand,
  UpdateCommand,
  DeleteCommand,
} = require("@aws-sdk/lib-dynamodb");
const { randomUUID } = require("crypto");

const TABLE = process.env.PRODUCTOS_TABLE;
const db = DynamoDBDocumentClient.from(new DynamoDBClient());

const respuesta = (statusCode, body) => ({
  statusCode,
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(body),
});

const leerBody = (event) => {
  try {
    return JSON.parse(event.body || "{}");
  } catch {
    return null;
  }
};

const validar = (data) => {
  if (!data) return "El cuerpo debe ser un JSON válido";
  if (typeof data.nombre !== "string" || !data.nombre.trim())
    return '"nombre" es obligatorio y debe ser texto';
  if (typeof data.precio !== "number" || data.precio < 0)
    return '"precio" es obligatorio y debe ser un número >= 0';
  if (data.stock !== undefined && !Number.isInteger(data.stock))
    return '"stock" debe ser un número entero';
  return null;
};

// CREATE - POST /productos
module.exports.crear = async (event) => {
  const data = leerBody(event);
  const error = validar(data);
  if (error) return respuesta(400, { error });

  const item = {
```

```

        id: randomUUID(),
        nombre: data.nombre,
        precio: data.precio,
        stock: data.stock ?? 0,
        creadoEn: new Date().toISOString(),
    });

    try {
        await db.send(new PutCommand({ TableName: TABLE, Item: item }));
        return respuesta(201, item);
    } catch (err) {
        console.error(err);
        return respuesta(500, { error: "No fue posible crear el producto" });
    }
};

// READ (todos) - GET /productos
module.exports.listar = async () => {
    try {
        const { Items } = await db.send(new ScanCommand({ TableName: TABLE }));
        return respuesta(200, Items);
    } catch (err) {
        console.error(err);
        return respuesta(500, { error: "No fue posible listar los productos" });
    }
};

// READ (uno) - GET /productos/{id}
module.exports.obtener = async (event) => {
    const { id } = event.pathParameters;
    try {
        const { Item } = await db.send(
            new GetCommand({ TableName: TABLE, Key: { id } })
        );
        if (!Item) return respuesta(404, { error: "Producto no encontrado" });
        return respuesta(200, Item);
    } catch (err) {
        console.error(err);
        return respuesta(500, { error: "No fue posible consultar el producto" });
    }
};

```

4. Reto: completar actualizar y eliminar

Cada grupo debe implementar las funciones actualizar y eliminar teniendo en cuenta lo siguiente:

Función	Pistas
actualizar	Usar UpdateCommand con UpdateExpression ("SET atributo = :valor, ..."), ExpressionAttributeValues y ReturnValues: "ALL_NEW" para devolver el registro actualizado. Validar el cuerpo igual que en crear.
eliminar	Usar DeleteCommand con la llave { id }.
Ambas	DynamoDB no falla si el id no existe: UpdateCommand crearía un registro nuevo y DeleteCommand no haría nada. Agregar ConditionExpression: "attribute_exists(id)" y, si se lanza el error ConditionalCheckFailedException, responder 404 .

Importante: algunos nombres de atributo son palabras reservadas de DynamoDB (por ejemplo name, status, date, year). Si su entidad los usa, deben declararse con ExpressionAttributeNames (por ejemplo #name).

5. Despliegue en AWS

```
serverless deploy
```

Al finalizar, la terminal muestra las URLs de los endpoints. Verificar en la consola de AWS:

- **Lambda:** que existan las 5 funciones (crud-productos-dev-crear, etc.) y su variable de entorno.
- **API Gateway:** que la HTTP API tenga las 5 rutas.
- **DynamoDB:** que la tabla crud-productos-productos-dev esté creada con id como partition key.
- **CloudFormation:** la pila crud-productos-dev y los recursos que generó.

6. Pruebas locales

A diferencia del laboratorio anterior, el código local necesita una tabla real. Por eso primero se despliega (paso 5) y luego se ejecuta el modo offline, que usa las credenciales locales para conectarse a la tabla de AWS:

```
serverless offline
```

Probar desde Postman o Insomnia, por ejemplo:

```
POST http://localhost:3000/productos
Content-Type: application/json

{
  "nombre": "Teclado mecánico",
  "precio": 120000,
  "stock": 5
}
```

Copiar el id retornado y probar el resto de rutas:

```
GET http://localhost:3000/productos
GET http://localhost:3000/productos/{id}
PUT http://localhost:3000/productos/{id}
DELETE http://localhost:3000/productos/{id}
```

7. Prueba en producción

Repetir el flujo completo con la URL generada por el despliegue, por ejemplo:

```
POST https://abcd1234.execute-api.us-east-1.amazonaws.com/productos
```

Se deben probar tanto los casos exitosos como los de error:

- Crear con un cuerpo inválido o sin campos obligatorios → **400**.
- Consultar, actualizar o eliminar un id inexistente → **404**.
- Después de cada operación, revisar en DynamoDB (opción **Explorar elementos**) que los datos cambiaron.
- Revisar en **CloudWatch Logs** los registros de al menos una de las funciones.

8. Limpieza de recursos

Al terminar la sustentación, eliminar todos los recursos para evitar costos:

Retos opcionales (bonificación)

- Agregar una ruta que busque por un atributo distinto al id usando un **Global Secondary Index** y `QueryCommand` (en lugar de `Scan`).
- Implementar paginación en `listar` con `Limit` y `LastEvaluatedKey`.
- Implementar `PATCH /productos/{id}` para actualizar solo los campos enviados.
- Ejecutar las pruebas locales con **DynamoDB Local** (Docker) sin usar la tabla de AWS.
- Construir un front sencillo (HTML + JavaScript) que consuma la API, habilitando CORS en `httpApi`.

Criterios de evaluación

Criterio	Peso
<code>serverless.yml</code> completo: 5 funciones, tabla DynamoDB, variable de entorno y permisos IAM limitados	20%
CRUD funcional con entidad propia (mínimo 4 atributos) desplegado en AWS	30%
Validaciones y códigos HTTP correctos (201, 400, 404, 500)	15%
Pruebas locales con <code>serverless-offline</code> y pruebas en AWS	15%
Verificación de recursos en la consola (Lambda, API Gateway, DynamoDB, CloudWatch)	10%
Documento técnico y reflexión	10%

Entrega

Cada grupo debe entregar:

1. **Video (máx. 7 minutos)** mostrando:
 - Explicación breve del `serverless.yml` y del `handler.js`.
 - Ejecución local con `serverless offline`.
 - Despliegue con `serverless deploy`.
 - Las 5 operaciones probadas en Postman/Insomnia contra AWS, incluyendo al menos un caso 400 y un caso 404.
 - Verificación de los datos en DynamoDB y de los logs en CloudWatch.
 - URL API
2. **Repositorio** con:
 - Código fuente (`handler.js`, `serverless.yml`, etc.).
 - Archivo `package.json`.
 - Colección de Postman/Insomnia exportada.
 - `README.md` con las instrucciones para desplegar y probar.
3. **Documento breve (1 a 2 páginas)** con:

- Descripción de la entidad elegida y del diseño de la tabla (partition key y atributos).
- Diagrama de la arquitectura.
- Respuestas a: ¿Por qué Scan puede ser costoso en tablas grandes y cuándo usar Query? ¿Qué ventajas tiene una función Lambda por operación frente a una sola Lambda con todas las rutas? ¿Qué pasaría si las funciones tuvieran permiso `dynamodb:*` sobre `*`?

FECHA: 27 septiembre 2026 11:59 PM